Whole-Body Model Predictive Control for Industrial Humanoids

Research prototype of whole-body model predictive control on the Unitree H1 in MuJoCo, with joint PD torque control, gravity compensation, and a control-barrier-function safety filter. The standing balance and the 80 N torso push are working. Lateral weight transfer onto one sole is working, and a swing can start after that transfer. One complete step does not yet finish in a stable double support. Walking is not solved.

Commands below are run from the `project` directory. The sections after the control loop walk through one tick and then each module. The running section lists every command-line argument.

What the robot is doing

The H1 model has a floating base, 19 torque actuators, and a single ankle-pitch joint on each foot. There is no ankle-roll actuator. When the body leans sideways the lower leg tilts, the sole capsule rolls, and the measured support region moves with the foot.

Two modes share the same MPC, PD controller, and CBF filter.

Standing (`--no-walk`). The robot holds its centre of mass over the middle of the two soles. At t = 2.0 s an 80 N force is applied to the torso in +x for 0.2 s. The filter intervenes and the stance recovers. A 6.5 s headless run stays in `STAND`, with a minimum pelvis height of about 0.927 m and a peak CBF intervention of about 120.27 N · m.

Walking (the default). The gait does not lift a foot until the centre of mass and the capture point are inside the stance sole. The demo always starts with `first_swing="right"`, which means:

1. stand on both feet
2. shift the centre of mass onto the **left** foot
3. swing the **right** foot forward
4. land the right foot
5. double support
6. if more steps remain, shift onto the right foot and swing the left foot

That is a left-foot-loaded first step, not a right-foot-loaded gait.

The current one-step runs shift successfully, keep the centre of mass inside the left sole, and swing the right foot for 0.8 s with the CBF still optimal. After the right foot lands, the body rolls off the outside of the left sole. Shortening the step, slowing the swing, slowing the joint ramp, and freezing the swing leg do not remove that landing failure. Do not treat a longer survival time as a solved step.

Control loop

```
MuJoCo state
    -> WholeBodyMPC          joint position targets from a CoM / torso task
    -> walking logic         lateral shift, swing IK, landing hold
    -> PD + gravity          tau = Kp (q* - q) + Kd (qd* - qd) + bias
    -> CBF safety filter     torque closest to the PD command that keeps
                             the centre of mass inside the support region
    -> MuJoCo
```

The PD torque is recomputed on every physics step. The MPC and the CBF are solved once per control tick, and the CBF's torque correction is held across the physics steps inside that tick.

Rate	Value
Physics step	0.002 s (500 Hz)
Control step	0.02 s (50 Hz), 10 physics steps per tick
MPC horizon	20 control steps (0.4 s at the default `dt`)

The MPC treats the centre of mass and the torso attitude as decoupled double integrators and turns the first-step task command into joint targets with a contact-relative Jacobian. It does not command the hip roll that the weight shift owns. Because of that, a centre-of-mass reference that sits more than about 8 mm away from the body saturates the lateral acceleration at ± 5 m/s^2 without moving the hip roll. The shift therefore keeps its reference inside that 8 mm band.

The CBF is a higher-order barrier on the axis-aligned box of the measured soles (`alpha1 = alpha2 = 12`). Torque corrections are limited to about 40 N · m. If the quadratic program is infeasible the filter repeats the last safe torque. Standing uses a 2 cm support margin. Walking sets that margin to 0, because a 2 cm margin puts the home centre of mass on the heel face.

How one tick works

`run_demo` in `main_demo.py` builds the stack, resets the robot, and then calls one function per control tick.

`--no-walk` calls `control_step` only. The reference is fixed at reset by `stand_reference`: torso attitude held, centre of mass held over the middle of the soles. At t = 2 s, `apply_push` writes a world-frame force onto the torso. MuJoCo clears `xfrc_applied` on every physics step, so the environment reapplies that wrench until the 0.2 s duration ends.

Walking calls `walk_tick`, which prepares the reference and the joint overrides and then calls the same `control_step`.

`control_step` does this, in order:

1. Read `env.get_state()` (centre of mass, joint positions and velocities, contact Jacobians, base rotation).
2. If a foot is swinging, keep only that stance foot's contact Jacobian. The MPC must not treat the moving foot as planted.
3. `WholeBodyMPC.solve` returns `q_des` and `qd_des`. The velocity target is zeros. The position target is the home pose plus one tick of the solved joint motion plus a capture offset.
4. If a `LateralWeightShift` is active, `apply` overwrites the leg targets. Hip roll follows the lean only. Hip pitch and knee keep a small MPC residual. Ankle pitch levels the sole and adds a heel term when the centre of mass has slid backward.
5. If a swing pose is present, `_compose_step_joints` writes it onto the swing leg only. The stance leg is left on the shift.
6. `PDController.compute_torque` adds MuJoCo's bias force (`qfrc_bias`) as gravity compensation and clips the result.
7. `CBFFilter.safe_torque` solves one quadratic program. The difference between the safe torque and the nominal torque is frozen for the next 10 physics steps.
8. `env.step_feedback` recomputes the PD torque on every 0.002 s step, adds that frozen correction, and steps MuJoCo.

`walk_tick` does this before `control_step`:

1. Ask `_com_over_foot` whether the loaded foot is ready. That is the swing gate.
2. `WalkingStateMachine.update` advances the phase only when the timer has elapsed and, for a shift, the gate is true.
3. Start from the state machine's centre-of-mass target. While a foot is loaded, replace that target with `LateralWeightShift.advance`. The shift reference stays within 8 mm of the current centre of mass.
4. Build the support polygon from the measured sole corners. During a swing it is the stance sole only, and only if the centre of mass is already inside it. Otherwise the polygon stays both feet and the swing is suppressed for that tick.
5. Solve `LegIK` for both feet. Apply it only to the swing leg, ramped by `--swing-ramp`.
6. When the swing ends, `_hold_landed_leg` copies the swing command that was actually reached into the MPC home pose for that leg's pitch, knee, and ankle. The hip roll of the landed leg stays on the lean blend so it does not snap back to the stand pose.

7. After the loaded foot becomes `None` (double support and the following stand), the shift posture stays active. Dropping it on the landing tick used to replace the 8 mm reference with the state machine's foot landmark, about 3 cm off the body, and remove the heel ankle term in the same tick.

`--landing-handoff` changes only that last stage. It freezes the touchdown reference instead of letting the 8 mm clamp follow the body outward, and it writes the landed pitch, knee, and ankle from the touchdown home pose so the MPC capture offset is not added on top. `--late-swing-capture` changes only `advance` during the swing: if the outer capture margin is under 5 cm and the body is moving outward, the lean blend may fall by at most 0.04, at 0.15 per second. Both flags default off.

Module guide

Read the files in this order if you are new to the repository.

`env/industrial_humanoid_env.py`

Gymnasium-style wrapper around the H1 scene. `EnvConfig.sim_dt` is 0.002 s and `control_dt` is 0.02 s. The class loads the MJCF, maps actuators to joints, and exposes `get_state`, `step_feedback`, and `apply_external_force`.

`get_state` is what every controller reads: centre-of-mass position and velocity, joint position and velocity, the torso rotation, and a contact Jacobian per foot. The centre of mass is MuJoCo's subtree centre of mass, not a point fixed in the pelvis.

`step_feedback` is the inner loop. A callback returns a torque each physics step. The demo's callback is the PD law plus the CBF correction. Do not send CBF torques through a path that holds one torque for the whole 0.02 s. The knees buckle if the PD term is not refreshed at 500 Hz.

`controllers/mpc_controller.py`

`WholeBodyMPC` regulates six task axes: centre-of-mass x, y, z and a rotation-vector torso error. Each axis is a double integrator. The horizon cost is diagonal, so the six problems are solved separately. Unconstrained axes use a precomputed least-squares inverse. An axis whose acceleration would leave the box $\pm$`max_com_acceleration` (5 m/s^2) is passed to `scipy.optimize.lsq_linear`.

Only the first acceleration of the horizon is used. It becomes a joint-velocity request through the contact-relative Jacobian `J_com - J_feet`, restricted to the actuated joints. The raw centre-of-mass Jacobian would spend the command on the unactuated floating base. Measured task velocity is not fed back. On a floating base that velocity is the base falling, and copying it into the joints makes the legs join the fall.

The position command returned to the PD controller is

```
q_des = q_home + qd * dt + capture_offset
```

`capture_offset` is a gain of 12 on the horizontal capture error, clipped to $\pm$0.15 rad, applied to the ankles in x and to both hip rolls in y. `qd_des` is zeros. The posture term pulls redundant joints toward the home pose recorded at construction.

This class does not know about steps. Walking is entirely outside it, in `walk_tick` and `LateralWeightShift`. The MPC docstring still describes a double-support posture controller because contact forces, friction, and swing scheduling are not inside the quadratic program. The CBF is what rejects centre-of-mass accelerations the feet cannot produce.

Weights that matter when you read a log: `w_com = 100`, `w_acceleration = 1e-3`. About 1 cm of lateral position error is enough to saturate the $\pm$5 m/s^2 box. That is why the shift reference is clamped to 8 mm.

`controllers/lateral_shift.py`

`LateralWeightShift` is the lateral actuator during a transfer. `begin` is called once per loaded foot. It solves a planted-ankle posture with `scipy.optimize.least_squares` on a private `MjData`, so the live simulation

is not touched. The target is the centre of the measured sole at that moment. A solve is accepted only when each ankle stays within 2 cm and the cost is under 0.05. The H1 ankles are about 40 cm apart at the home pose. A one-sided hip-roll command cannot cover that distance, because both hip-roll axes point the same way and the MPC capture clip on hip roll is only ±0.15 rad.

`advance` moves a blend fraction from the home pose toward that solution. The fraction increases at 0.05 per second while the capture point is short of a hold point 1.5 cm inside the inner sole edge. It brakes when the body is already moving onto the sole. Once the centre of mass is inside the sole and the lateral speed is under 0.03 m/s, the fraction latches (`limit = "com_inside"`) so the swing cannot unwind the lean. Reference y is the blend, then clipped to the body ±8 mm. Reference x stays at the double-support origin, also clipped to ±8 mm. Pulling x toward the sole centre walks the body onto the toes.

`apply` writes the blended joints over the MPC command. Hip roll has no MPC residual. Hip pitch and knee keep ±0.10 rad of the MPC command. Ankle pitch is `-(hip_pitch + knee)` plus up to ±0.15 rad of ankle capture plus a heel term `clip(-4 * (origin_x - com_x), -0.2, 0.05)`. That heel term is what holds the rear support edge open during the lean. It lives only in `apply`. If the posture is cleared, the term disappears on that tick.

`skip_side` leaves the landed leg's pitch, knee, and ankle alone. Hip roll on that leg still follows the lean. `support_distance` is zero inside the sole rectangle and the Euclidean distance to the box otherwise.

`controllers/walk_state_machine.py`

`WalkingStateMachine.update(dt, support_ready)` is the clock the demo follows. `sample` is a separate open-loop schedule used by older callers. Do not mix them. `build_step_command` is not what `walk_tick` uses.

`loaded_foot` is `left` in `SHIFT_LEFT` and `RIGHT_SWING`, `right` in `SHIFT_RIGHT` and `LEFT_SWING`, and `None` in `STAND` and `DOUBLE_SUPPORT`. The swing gate is evaluated on `loaded_foot`. A shift will not leave until its duration has elapsed and `support_ready` is true. Swing, double support, and stand leave on the timer alone.

`next_state` encodes the support rule: `RIGHT_SWING` is reached only from `SHIFT_LEFT`, and `LEFT_SWING` only from `SHIFT_RIGHT`. After the last step, `DOUBLE_SUPPORT` returns to `STAND`, and `STAND` aims the centre of mass at the midpoint of the two stored feet. During `DOUBLE_SUPPORT` the target stays on the foot that just carried the step.

The demo's `WalkConfig` is not the dataclass defaults. `run_demo` sets stand 1.5 s, shift 1.5 s, swing from `--step-duration` (0.8 s), double support 0.4 s, step height 0.05 m, and `first_swing="right"`. A stand shorter than about 1 s before the lean starts makes the robot fall. The regression tests use a 1.0 s stand for that reason.

`controllers/footstep_planner.py`

`FootstepPlanner` places the next landing `step_length` ahead of the opposite foot along +x. `SwingFootTrajectory` splits the swing into three equal phases (lift, move forward, land) and fits a C1 cubic on each. Velocity is zero at liftoff, at the phase junctions, and at touchdown. The state machine builds this trajectory itself inside `_emit`. The planner class is what the regression test checks.

`controllers/leg_ik.py`

`LegIK.solve` closes hip pitch, knee, and ankle pitch for one foot target. Thigh and shank are each 0.4 m. Hip yaw and hip roll are not solved. They stay at the current angle and define the plane the pitch joints move in. At the home pose the solution is hip pitch −0.4 rad, knee +0.8 rad, ankle pitch −0.4 rad, which is `ankle = -(hip_pitch + knee)` with an upright pelvis. An unreachable target is flagged on `LegJoints.reachable`.

controllers/pd_controller.py

`compute_torque` is

```
tau = Kp (q_des - q) + Kd (qd_des - qd) + tau_ff
```

clipped to ± 200 N · m. Legs use Kp 400 and Kd 40, the torso 300 and 30, the arms 60 and 3. The demo passes MuJoCo's `qfrc_bias` as `tau_ff`, which is the gravity and Coriolis compensation. Ankle motors in the MJCF are still limited to about ± 40 N · m even though the PD clip is 200.

safety/cbf_filter.py

`CBFFilter.safe_torque` minimises `||u - u_nominal||` subject to a higher-order barrier on each face of the support box and a trust region of `max_torque_deviation = 40` N · m. The torque-to-acceleration map `B` is linearised once at the home double-support pose. It is wrong once a foot is in the air, which is why a large swing step used to make the filter fail safe and hold the previous torque. The swing ramp exists so the PD step stays inside that trust region.

`set_support_polygon` replaces the box with the axis-aligned bounds of the measured sole corners. The barrier is not a general polygon. `com_barrier_values` returns signed distances: `support_x_lower = com_x - rear_edge`, positive when the centre of mass is inside. Status strings are `optimal`, `relaxed`, and `fail_safe`.

Walking sets `cbf.config.margin = 0`. The standing default is 0.02 m.

swing_diagnostic.py

`summarize_single_support` reads the in-memory rows from a walking run and prints the first swing. It does not invent numbers. `save_support_plot` writes the five-panel landing figure. Both are called from `_finish_log` only when `args.walk` is set, so a `--no-walk` run does not print a swing report.

visualize_demo.py

A fixed 6.5 s standing run with the same `build_stack`, `control_step`, and push as `--no-walk`. It writes figures under `data/figures/` and does not open a viewer. It does not call `run_demo` or `parse_args`. Extra demo flags do not affect it.

rl/safe_rl_training.py

A Lagrangian PPO-style trainer whose policy adds a residual to the MPC reference. The CBF still filters torques unless `--no-cbf` is set. Nothing in `main_demo.py` imports this module. Training it is not part of the walking experiments.

tests/test_lateral_shift.py

Six checks, all required to keep passing:

Check	What it asserts
`planners`	Four alternating steps land at 0.1, 0.2, 0.3, 0.4 m, and the cubic reaches the step height. The state machine can enter `SHIFT_LEFT` and `RIGHT_SWING` when the gate is forced true.
`leg_ik`	Home ankles solve to hip pitch -0.4, knee 0.8, ankle -0.4.
`solver`	The shift target is the measured sole centre, and the home-ankle solve is accepted.
`stand`	Two seconds of `control_step` stay above 0.85 m and finish within 5 cm of the reference in x.

Check	What it asserts
`walk_stays_up`	After a 1 s stand and 200 ticks, the robot is in `SHIFT_LEFT`, has not swung, and stays above 0.85 m.
`target_moves`	Over 350 ticks the reference y moves more than 5 cm toward the left sole, the swing stays down, and the logged stance target matches the solver.

Walking behaviour

States, in order for the default first step:

`STAND` → `SHIFT_LEFT` → `RIGHT_SWING` → `DOUBLE_SUPPORT` → `SHIFT_RIGHT` → `LEFT_SWING` → `DOUBLE_SUPPORT` → …

Phase	What is held
`STAND`	Both feet down. Centre-of-mass target between the feet. 1.5 s before the first shift.
`SHIFT_LEFT` / `SHIFT_RIGHT`	Both feet stay down. `LateralWeightShift` leans the body onto the measured sole. The other foot cannot lift.
`RIGHT_SWING` / `LEFT_SWING`	Stance foot is the one just loaded. The swing foot follows a cubic lift. Inverse kinematics is applied only to the swing leg, and the joint target moves at most `--swing-ramp` radians per tick.
`DOUBLE_SUPPORT`	Both feet down for 0.4 s. The centre of mass stays associated with the foot that just carried the step.

A shift lasts at least 1.5 s and does not end until the swing gate is true. The gate is unchanged by the diagnostic flags. A foot may lift only when all of these are true:

- the centre of mass is inside the stance sole, inset by 1 cm
- the capture point `com + velocity / sqrt(g / height)` is inside that same inset sole
- lateral speed is below 0.03 m/s
- forward speed is below 0.08 m/s
- the shift duration has elapsed

Default step geometry, also the values used when the flags are omitted:

Quantity	Default
Step length	0.10 m
Step height	0.05 m
Swing duration	0.8 s
Swing joint ramp	0.03 rad per control tick
Steps	4

`--late-swing-capture` and `--landing-handoff` are off unless you pass them. They are experiments around the landing, not a working gait. With both off, the one-step run still falls sideways after touchdown.

Layout

```
README.md
.venv/                                  Python 3.10 virtualenv (repo root)
project/
  main_demo.py                          Viewer or headless demo
  visualize_demo.py                     Headless push recovery and figures
  swing_diagnostic.py                   One-step swing / landing report and plot
  requirements.txt
  env/industrial_humanoid_env.py        MuJoCo H1 environment
  controllers/
    mpc_controller.py                   Whole-body MPC
    pd_controller.py                    Joint PD and gravity feedforward
    lateral_shift.py                    Planted-foot lean onto one sole
    walk_state_machine.py               STAND / shift / swing / double support
    footstep_planner.py                 Step length and cubic swing trajectory
    leg_ik.py                           Analytical swing-leg IK
  safety/cbf_filter.py                  Support-region CBF
  rl/safe_rl_training.py                Separate residual-RL trainer (not used by the demo)
  tests/test_lateral_shift.py           Six regression checks
  utils/                                Logging and plotting helpers
  data/                                 Logs, npz/csv traces, support plots, figures
  models/unitree_h1/                    Unitree H1 MJCF and meshes (scene.xml)
```

Setup

From the repository root, with Python 3.10:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r project\requirements.txt
```

The H1 scene must already be at `project/models/unitree_h1/scene.xml`. The demo does not download it.

Then change into the project directory. Every command below assumes that as the working directory, and the virtualenv interpreter:

```
cd project
..\.venv\Scripts\python.exe main_demo.py --help
```

Running the demo

`main_demo.py` parses arguments with `argparse`. `--walk` / `--no-walk` is a Boolean optional flag and defaults to walking. There is no `--render` flag. Opening the window is the default. `--headless` turns the window off.

```
..\.venv\Scripts\python.exe main_demo.py --help
```

Arguments

Argument	Default	Meaning
`--horizon`	20	MPC horizon in control steps.
`--control-dt`	0.02	Control period in seconds. Must be a positive integer multiple of the 0.002 s physics step.

Argument	Default	Meaning
`--seed`	`0`	Reset seed.
`--walk` / `--no-walk`	walk	`--walk` runs the gait. `--no-walk` stands and applies the torso push. Do not pass both.
`--steps`	`4`	Number of swing steps. This is not a simulation-step count. Use `--steps 1` for a single right-foot swing after the left-foot shift.
`--walk-speed`	`0.05`	Metres per second. Retained for the old planted-foot reference. The stepping gait does not use it.
`--push-force FX FY FZ`	`80 0 0`	World-frame force in newtons, applied at t = 2 s only in `--no-walk`. Duration is fixed at 0.2 s.
`--model-path`	`models/unitree_h1/scene.xml`	MJCF scene.
`--log-path`	`data/demo_log.npz`	Npz log. A csv with the same stem is written beside it.
`--step-length`	`0.1`	Forward step length in metres.
`--step-duration`	`0.8`	Swing duration in seconds.
`--swing-ramp`	`0.03`	Largest change in a swing-joint target in one control tick, in radians.
`--freeze-swing`	off	Still shifts and enters single support, but does not move the swing leg. Diagnostic only.
`--late-swing-capture`	off	During the swing, ease the lean by at most 0.04 of the blend, and only while the capture point is inside 5 cm of the outer sole edge and moving outward.
`--landing-handoff`	off	After touchdown, keep the touchdown centre-of-mass reference instead of letting it follow the body, and keep the landed pitch, knee, and ankle on the command the swing actually reached.
`--headless`	off	No viewer. Run for `--duration` seconds, then print the swing diagnostic when walking. Stops early if the pelvis drops below 0.72 m.
`--duration`	`24`	Simulated seconds for `--headless` only. Ignored when the viewer is open. The viewer runs until the window is closed.

Standing and the push

```
..\.venv\Scripts\python.exe main_demo.py --no-walk
```

Headless, matching the regression check:

```
..\.venv\Scripts\python.exe main_demo.py --no-walk --headless --duration 6.5 --log-path data/nowalk_che
```

Change the push without changing its timing:

```
..\.venv\Scripts\python.exe main_demo.py --no-walk --push-force 80 0 0
```

Walking in the viewer

Four steps, default length and timing. Close the window to stop and write the log.

```
..\.venv\Scripts\python.exe main_demo.py
..\.venv\Scripts\python.exe main_demo.py --steps 4 --step-length 0.1 --step-duration 0.8
```

One step:

```
..\.venv\Scripts\python.exe main_demo.py --steps 1
```

Headless one-step experiments

These keep the default step length, swing duration, and ramp. Each writes an npz, a csv, a text diagnostic on stdout, and a `*_support.png` plot next to the log.

```
..\.venv\Scripts\python.exe main_demo.py --headless --steps 1 --duration 22 --log-path data/swing_baseli

..\.venv\Scripts\python.exe main_demo.py --headless --steps 1 --duration 22 --step-duration 1.2 --log-pa

..\.venv\Scripts\python.exe main_demo.py --headless --steps 1 --duration 22 --step-length 0.05 --log-pa

..\.venv\Scripts\python.exe main_demo.py --headless --steps 1 --duration 22 --freeze-swing --log-path da

..\.venv\Scripts\python.exe main_demo.py --headless --steps 1 --duration 22 --swing-ramp 0.01 --log-patl

..\.venv\Scripts\python.exe main_demo.py --headless --steps 1 --duration 22 --late-swing-capture --log-p

..\.venv\Scripts\python.exe main_demo.py --headless --steps 1 --duration 22 --landing-handoff --log-patl

..\.venv\Scripts\python.exe main_demo.py --headless --steps 1 --duration 22 --late-swing-capture --landi
```

`--step-duration`, `--step-length`, and `--swing-ramp` were varied to test whether swing momentum caused the fall. They did not. Leave them at the defaults for the landing work. `--freeze-swing` is not a gait. `--late-swing-capture` and `--landing-handoff` do not yet produce a stable landing.

Push-recovery figures

`visualize_demo.py` does not take arguments. It runs the same standing stack headless for 6.5 s, including the 80 N push, and writes figures under `data/figures/`.

```
..\.venv\Scripts\python.exe visualize_demo.py
```

Tests

```
..\.venv\Scripts\python.exe -m tests.test_lateral_shift
```

The six checks cover the footstep planner, leg IK, the weight-shift solver, a 2 s stand, a walk that stays upright without swinging, and a centre-of-mass reference that moves toward the stance sole without lifting the other foot. The expected last line is `6 passed`.

Safe RL trainer

`rl/safe_rl_training.py` is a separate residual-policy trainer. The walking demo does not call it. From `project`:

```
..\.venv\Scripts\python.exe -m rl.safe_rl_training --help
```

Argument	Default	Meaning
`--algorithm`	`ppo_lagrangian`	`ppo_lagrangian`, `cpo`, or `sac_lagrangian`.
`--total-steps`	2000000	Training steps.
`--cost-budget`	5	Constraint on expected cumulative safety cost.
`--seed`	0	
`--run-dir`	`data/runs`	
`--no-cbf`	off	Train without the CBF filter.

Logs

On exit, walking or standing, the demo writes:

- `LOG.npz` with time, centre of mass, reference, error, velocity, CBF intervention, CBF status, ankle positions, walk state, support foot, stance-foot xy, and support distance
- `LOG.csv` with the same columns in text

A walking run also prints a single-support report and saves `LOG_support.png`. The figure covers the swing and the landing window: centre of mass and capture point against the rear and outer support edges, velocity, stance hip-roll target, ankle correction, capture margins, and CBF intervention. Vertical lines mark swing start, touchdown, the first non-optimal CBF solve, and the first time the rear margin reaches zero.

The printed report includes swing side and duration, the first rear-margin breach, the first non-optimal CBF status, minimum rear and capture margins, peak swing torque, and a touchdown table from 0.2 s before landing through 0.4 s after it. The numbers in that table come from the log. A run that never swings prints `No swing occurred.`

Current limit

Standing, the 80 N push, the shift onto the left sole, and the right swing are in place. The failure that remains is after the right foot touches down: lateral speed grows while the measured sole is still reported under the centre of mass, and the robot falls off the outside of the left foot. Do not paper over that by widening torque limits, the acceleration box, the CBF trust region, or the support polygon. Multi-step walking waits until one step stands up again in double support.